INFORMATION RETRIEVAL
PRACTICAL VIVA PREPARATION

Text Preprocessing using NLTK | SMS Spam Filtering using Naive Bayes and SVM

This guide explains both programs step by step, defines the important concepts, provides viva questions with answers, and includes short explanations you can speak to an external examiner.

PRACTICAL 1: TEXT PREPROCESSING USING NLTK

1. Aim

To perform text preprocessing using Natural Language Processing (NLP) techniques such as case folding, text cleaning, tokenization, stop-word removal and stemming using the NLTK library.

2. What does this practical do?

A computer needs text to be cleaned and organized before many language-processing tasks. This practical takes an input paragraph, converts it into lowercase, removes punctuation and other unwanted characters, splits it into tokens, removes common stop words, and compares two stemming algorithms: Porter and Lancaster.

3. Code explanation, step by step

Step 1: Import libraries

```
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, LancasterStemmer
from nltk.tokenize import word_tokenize
```

Import	Purpose
re	Python regular-expression module; used for pattern-based text cleaning.
nltk	Natural Language Toolkit, a Python library for NLP.
stopwords	Provides lists of common words that can be removed from text.
PorterStemmer	Implements the Porter stemming algorithm.
LancasterStemmer	Implements the Lancaster stemming algorithm.
word_tokenize	Splits text into individual tokens, usually words and punctuation.

Step 2: Download NLTK resources

```
nltk.download('punkt', quiet=True)
nltk.download('punkt_tab', quiet=True)
nltk.download('stopwords', quiet=True)
```

- punkt: tokenizer data used by NLTK.
- punkt_tab: additional tokenizer data required by some NLTK versions.
- stopwords: English stop-word list.
- quiet=True: suppresses most download messages.

If NLTK reports that a resource is missing, download the resource shown in the error message. These downloads generally need an internet connection.

Step 3: Define the input text

```
text = (
    "Information Retrieval systems are designed to retrieve relevant documents "
    "from large collections. Users are searching, browsing and ranking "
    "the retrieved results every day!"
)
```

The program stores a sample paragraph in the variable text. Adjacent string literals inside parentheses are automatically joined by Python.

Step 4: Case folding and cleaning

```
clean = re.sub(r'[^a-zA-Z\s]', ' ', text.lower())
```

text.lower() converts all letters to lowercase. re.sub(pattern, replacement, text) replaces characters matching a regular-expression pattern. In [^a-zA-Z\s], the caret means “not” because it appears immediately after the opening square bracket. The pattern therefore keeps English letters and whitespace; other characters, such as punctuation and digits, are replaced with a space.

Example: “Hello! World123” becomes “hello world”.

Step 5: Tokenization

```
tokens = word_tokenize(clean)
print('Tokens:', tokens)
```

Tokenization is the process of splitting text into smaller units called tokens. For example, “information retrieval systems” becomes [“information”, “retrieval”, “systems”].

Step 6: Stop-word removal

```
stop_words = set(stopwords.words('english'))
filtered = [t for t in tokens if t not in stop_words]
```

Stop words are common words that may contribute relatively little to a particular text-analysis task, such as “is”, “are”, “the”, “a” and “and”. The code loads the English stop-word list and uses a list comprehension to keep only tokens that are not in that list. A set is used because membership checking is efficient.

Example: [“users”, “are”, “searching”, “for”, “documents”] becomes [“users”, “searching”, “documents”]. Note: stop-word removal is task-dependent; some words that are commonly removed can be important in certain applications.

Step 7: Stemming with Porter and Lancaster

```
porter = PorterStemmer()
lancaster = LancasterStemmer()

for w in filtered:
    print(w, porter.stem(w), lancaster.stem(w))
```

Stemming reduces related word forms to a stem by applying rules, often removing suffixes. The resulting stem is not always a valid dictionary word.

Algorithm	Explanation
Porter Stemmer	Uses a sequence of suffix-removal rules. It is generally less aggressive than Lancaster.
Lancaster Stemmer	Uses a more aggressive stemming algorithm and can produce shorter stems.

Illustrative examples: “searching” may become “search”; “retrieved” may become “retriev”. Exact output depends on the algorithm. The program prints the original filtered word beside its Porter and Lancaster stems.

In the formatted print statement, <14 reserves a field width of 14 characters and aligns the columns. The f-string inserts calculated values into the output.

Step 8: Statistics

```
print("Tokens before :", len(tokens))
print("Tokens after  :", len(filtered))
print("Reduction     : {:.1f}%".format(
```

```
    100 * (len(tokens) - len(filtered)) / len(tokens)
))
print("Unique stems  :", len({porter.stem(w) for w in filtered}))
```

len(tokens) counts tokens before stop-word removal. len(filtered) counts tokens after removal. The percentage reduction is calculated as:

Reduction (%) = ((Tokens before − Tokens after) / Tokens before) × 100

The expression {porter.stem(w) for w in filtered} creates a set of Porter stems. A set keeps only unique values, so len(...) gives the number of unique Porter stems.

The input paragraph is non-empty, so division by len(tokens) is safe for this particular program. For a general-purpose program, check that len(tokens) is not zero before dividing.

4. Practical 1 workflow

Order	Operation	What happens
1	Input text	A sample paragraph is provided.
2	Case folding and cleaning	Text is converted to lowercase and unwanted characters are replaced.
3	Tokenization	Text is split into individual tokens.
4	Stop-word removal	Common English stop words are filtered out.
5	Stemming	Porter and Lancaster stems are generated and compared.
6	Statistics	Token counts, reduction percentage and unique stems are displayed.

5. Practical 1 viva questions and answers

Q. What is NLP?

Answer: Natural Language Processing is a branch of AI that enables computers to process, analyse and work with human language.

Q. What is NLTK?

Answer: NLTK stands for Natural Language Toolkit. It is a Python library for NLP tasks such as tokenization, stemming and stop-word removal.

Q. What is text preprocessing?

Answer: It is the process of cleaning and preparing raw text before further analysis or modelling.

Q. What is case folding?

Answer: Converting text into a consistent letter case, usually lowercase.

Q. What is tokenization?

Answer: Splitting text into smaller units called tokens, such as words or sentences.

Q. What is a token?

Answer: A unit of text produced by tokenization, commonly a word or punctuation mark.

Q. What are stop words?

Answer: Common words that may be less informative for a particular task, for example “the”, “is” and “and”.

Q. Why do we remove stop words?

Answer: To reduce less informative words and sometimes reduce the size of the text representation. It is not appropriate for every task.

Q. What is stemming?

Answer: Reducing words to a common stem by applying rules such as removing suffixes.

Q. Is a stem always a meaningful English word?

Answer: No. A stem can be a shortened form that is not a dictionary word.

Q. What is Porter Stemmer?

Answer: A rule-based stemming algorithm that applies a sequence of suffix-removal rules.

Q. What is Lancaster Stemmer?

Answer: A more aggressive stemming algorithm that can shorten words more than Porter.

Q. What is the difference between stemming and lemmatization?

Answer: Stemming applies rules to cut word endings. Lemmatization uses vocabulary and linguistic information to find a meaningful dictionary base form.

Q. What is regular expression (RegEx)?

Answer: A pattern used to match, search, split or replace text.

Q. What does re.sub() do?

Answer: It replaces parts of a string that match a regular-expression pattern.

Q. What does text.lower() do?

Answer: It converts alphabetic characters in the string to lowercase.

Q. What does \s mean in the regular expression?

Answer: It matches whitespace characters such as spaces, tabs and newlines.

Q. Why is a set used for stop words?

Answer: A set supports efficient membership checking when deciding whether a token is a stop word.

Q. What does len() do?

Answer: It returns the number of items in a sequence or collection.

Q. What are unique stems?

Answer: The distinct stemmed forms remaining after duplicate stems are counted only once.

PRACTICAL 2: SMS SPAM FILTERING USING NAIVE BAYES VS SVM

1. Aim

To build an SMS spam detection system using TF-IDF feature extraction and compare Multinomial Naive Bayes and Linear Support Vector Machine (SVM) classifiers.

2. What does this practical do?

The program learns from SMS messages that are already labelled as spam or ham. It converts the messages into numerical features using TF-IDF, trains two classification algorithms, evaluates their predictions, and uses the trained Naive Bayes model to classify new messages.

HAM means a genuine or non-spam message. SPAM means an unwanted message, often promotional or fraudulent.

3. Code explanation, step by step

Step 1: Import libraries

```
import pandas as pd
import urllib.request
import zipfile
import io
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import LinearSVC
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix)
```

Library / function	Purpose
pandas	Loads and manages tabular data.
urllib.request	Downloads the dataset from a URL.
zipfile	Reads and extracts ZIP archives.
io.BytesIO	Treats downloaded bytes as a file in memory.
train_test_split	Divides data into training and testing sets.
TfidfVectorizer	Converts text into numerical TF-IDF features.
MultinomialNB	Implements the Multinomial Naive Bayes classifier.
LinearSVC	Implements a linear Support Vector Classifier.
sklearn.metrics	Provides functions to evaluate classification results.

Step 2: Download and load the dataset

```
url = "https://archive.ics.uci.edu/static/public/228/sms+spam+collection.zip"
with urllib.request.urlopen(url) as response:
    zip_data = response.read()
with zipfile.ZipFile(io.BytesIO(zip_data)) as z:
    with z.open("SMSSpamCollection") as f:
        df = pd.read_csv(f, sep="\t", header=None,
                         names=["label", "text"])
```

The URL points to the SMS Spam Collection dataset hosted by the UCI Machine Learning Repository. urlopen() opens the URL and response.read() downloads its contents. ZipFile() opens the ZIP archive, and z.open() opens the file named SMSSpamCollection inside it.

- sep="\t": the file uses a tab character to separate the label and message.
- header=None: the source file does not contain a header row.
- names=["label", "text"]: assigns column names to the two fields.

io.BytesIO(zip_data) lets the ZIP archive be read directly from memory, so the code does not need to save the ZIP file to disk first.

Step 3: Convert labels into numbers

```
df['y'] = (df['label'] == 'spam').astype(int)
```

The model target is converted into numeric form: spam becomes 1 and ham becomes 0. The comparison returns True for spam and False for ham; astype(int) converts True to 1 and False to 0.

```
print("Dataset size:", df.shape[0],
      "| spam:", df["y"].sum(),
      "| ham:", (1 - df["y"]).sum())
```

- df.shape[0] gives the number of rows/messages.
- df["y"].sum() counts spam messages because spam is represented by 1.
- (1 - df["y"]).sum() counts ham messages because ham is represented by 0.

The standard SMS Spam Collection dataset contains 5,574 messages: 747 spam and 4,827 ham.

Step 4: Split into training and testing sets

```
X_train, X_test, y_train, y_test = train_test_split(
    df['text'], df['y'], test_size=0.2,
    random_state=42, stratify=df['y'])
```

Variable / setting	Meaning
X_train	SMS message text used for training.
X_test	SMS message text held out for testing.
y_train	Correct labels for the training messages.
y_test	Correct labels for the testing messages.
test_size=0.2	Uses 20% of the data for testing and 80% for training.
random_state=42	Makes the split reproducible when the data and software setup are unchanged.
stratify=df["y"]	Maintains approximately the same ham/spam class proportions in both sets.

Training data teaches the model. Testing data checks how well it predicts messages it did not train on. Keeping the test set separate gives a more realistic evaluation of generalization.

Step 5: TF-IDF feature extraction

```
vec = TfidfVectorizer(
    lowercase=True,
    stop_words='english',
    ngram_range=(1, 2),
    min_df=2
)
Xtr = vec.fit_transform(X_train)
Xte = vec.transform(X_test)
```

TF-IDF stands for Term Frequency–Inverse Document Frequency. It converts text into numerical feature values. It gives a word more weight when it is important in a document but not common across all documents.

A simplified description of the components:

- Term Frequency (TF): how frequently a term occurs in a document.
- Inverse Document Frequency (IDF): reduces the weight of terms that occur in many documents.

Parameter	Meaning
lowercase=True	Converts text to lowercase before feature extraction.
stop_words="english"	Removes common English stop words.
ngram_range=(1, 2)	Uses unigrams (single words) and bigrams (pairs of consecutive words).
min_df=2	Ignores terms that occur in fewer than two documents in the training corpus.

For “win a free prize”, unigrams include “win”, “a”, “free” and “prize”; bigrams include “win a”, “a free” and “free prize”.

fit_transform(X_train) learns the vocabulary and IDF weights from the training messages, then converts them into a feature matrix. transform(X_test) converts the test messages using the same learned vocabulary and weights. Fitting only on training data avoids leaking information from the test set into training.

Xtr.shape displays the number of training messages and the number of TF-IDF features.

Step 6: Create the classifiers

```
models = {
    'Multinomial Naive Bayes': MultinomialNB(alpha=0.1),
    'Linear SVM': LinearSVC()
}
```

A. Multinomial Naive Bayes

Naive Bayes is a probabilistic classifier based on Bayes’ theorem. It estimates the probability that a message belongs to a class, such as spam or ham. The “naive” assumption is that features are conditionally independent given the class.

Bayes’ theorem: $P(A|B) = [P(B|A) \times P(A)] / P(B)$. In this practical, the model uses the words/features in a message to estimate whether the message is spam or ham.

alpha=0.1 is the smoothing parameter. Smoothing helps avoid zero probability for a feature that was not observed in a particular class during training.

B. Linear Support Vector Machine (SVM)

SVM is a supervised learning algorithm used for classification. It finds a decision boundary, called a hyperplane, that separates classes. A linear SVM uses a straight-line decision boundary in two dimensions, or its higher-dimensional equivalent in a larger feature space.

Support vectors are the training points closest to the decision boundary. They influence the position of the boundary. LinearSVC() implements a linear support vector classifier.

Naive Bayes	Linear SVM
Probabilistic classifier based on Bayes’ theorem.	Margin-based classifier that learns a separating hyperplane.
Uses a conditional-independence assumption.	Does not use the Naive Bayes independence assumption.
Often fast and a useful text-classification baseline.	Often effective with high-dimensional sparse text features.
MultinomialNB uses alpha for smoothing.	LinearSVC learns a linear decision boundary.

Step 7: Train and predict

```
for name, clf in models.items():
    clf.fit(Xtr, y_train)
    pred = clf.predict(Xte)
```

models.items() gives each model name and classifier. fit(Xtr, y_train) trains the classifier using training features and labels. predict(Xte) predicts labels for the test features. The predicted labels are stored in pred.

Step 8: Evaluate the models

```
accuracy_score(y_test, pred)
precision_score(y_test, pred)
recall_score(y_test, pred)
f1_score(y_test, pred)
```

Metric	Meaning	Formula
Accuracy	Fraction of all predictions that are correct.	(TP + TN) / (TP + TN + FP + FN)
Precision	Of messages predicted as spam, the fraction that really are spam.	TP / (TP + FP)
Recall	Of actual spam messages, the fraction correctly detected.	TP / (TP + FN)
F1-score	Harmonic mean of precision and recall.	2 × (Precision × Recall) / (Precision + Recall)

round(value, 4) rounds a metric to four decimal places. Precision and recall use spam (label 1) as the positive class by default in this code.

Step 9: Understand the confusion matrix

```
confusion_matrix(y_test, pred, labels=[0, 1])
```

With labels=[0, 1], the matrix is arranged as [[TN, FP], [FN, TP]]. Rows represent actual labels and columns represent predicted labels.

	Predicted Ham (0)	Predicted Spam (1)
Actual Ham (0)	True Negative (TN)	False Positive (FP)
Actual Spam (1)	False Negative (FN)	True Positive (TP)

- TP: spam correctly classified as spam.
- TN: ham correctly classified as ham.
- FP: genuine ham incorrectly classified as spam.
- FN: spam incorrectly classified as ham.

A confusion matrix shows the kinds of errors made by a classifier, not just its overall accuracy.

Step 10: Classify new messages

```
nb = models['Multinomial Naive Bayes']
new_msgs = [
    'WINNER!! You have won a free prize. Call now to claim',
    'Hi, are we still meeting for the project discussion tomorrow?',
    'Congratulations! Click the link to claim your FREE gift voucher today'
]
for m, p in zip(new_msgs, nb.predict(vec.transform(new_msgs))):
    print('[SPAM]' if p else '[HAM]', m)
```

The code selects the already-trained Naive Bayes model. vec.transform(new_msgs) converts the new messages using the same TF-IDF vectorizer fitted on training data. predict() produces a numeric label for each message. zip() pairs each message with its prediction. The conditional expression displays SPAM for 1 and HAM for 0.

Important: both Naive Bayes and SVM are trained and evaluated, but only the Naive Bayes model is used for the final new-message predictions in this code.

4. Practical 2 workflow

Order	Operation	What happens
1	Dataset loading	Download and read the UCI SMS Spam Collection dataset.
2	Label encoding	Convert spam to 1 and ham to 0.
3	Train-test split	Use 80% for training and 20% for testing.
4	TF-IDF	Convert messages into numerical features.
5	Model training	Train Multinomial Naive Bayes and Linear SVM.
6	Evaluation	Compare accuracy, precision, recall, F1-score and confusion matrices.
7	New predictions	Use the trained Naive Bayes model to classify new messages.

5. Worked example of evaluation metrics

Suppose a test set contains 100 messages: 20 spam and 80 ham. The classifier correctly identifies 18 spam messages and 75 ham messages. It incorrectly marks 5 ham messages as spam and misses 2 spam messages.

Value	Count	Explanation
TP	18	Spam correctly predicted as spam.
TN	75	Ham correctly predicted as ham.
FP	5	Ham incorrectly predicted as spam.
FN	2	Spam incorrectly predicted as ham.

Accuracy = (18 + 75) / 100 = 93%.

Precision = 18 / (18 + 5) = 0.7826, or 78.26%.

Recall = 18 / (18 + 2) = 0.90, or 90%.

F1-score = 2 × (0.7826 × 0.90) / (0.7826 + 0.90) ≈ 0.8372.

This is an illustrative example to explain the formulas; it is not a claim about the exact output of your program.

6. Practical 2 viva questions and answers

Q. What is the objective of this practical?

Answer: To classify SMS messages as spam or ham using machine learning and compare Naive Bayes with Linear SVM.

Q. What is spam filtering?

Answer: Identifying unwanted messages and separating them from genuine messages.

Q. What do spam and ham mean?

Answer: Spam is an unwanted message; ham is a genuine, non-spam message.

Q. Which type of machine learning is used?

Answer: Supervised learning, because the training dataset contains labelled messages.

Q. What is classification?

Answer: A supervised learning task that assigns an input to one of a set of predefined classes.

Q. What is UCI?

Answer: The University of California, Irvine Machine Learning Repository, which provides datasets for research and education.

Q. How many messages are in the standard SMS Spam Collection?

Answer: 5,574 messages: 4,827 ham and 747 spam.

Q. Why are labels converted to numbers?

Answer: The target classes are encoded as 0 and 1 so the model and evaluation functions can work with numeric labels.

Q. What is feature extraction?

Answer: Converting raw data into useful numerical features that a machine learning model can process.

Q. What is TF-IDF?

Answer: Term Frequency–Inverse Document Frequency; a method for converting text into numerical weights based on term frequency and document frequency.

Q. Why is TF-IDF used?

Answer: Machine learning classifiers need numerical input; TF-IDF represents the importance of terms as numbers.

Q. What is Term Frequency?

Answer: A measure of how often a term occurs in a document.

Q. What is Inverse Document Frequency?

Answer: A measure that reduces the weight of terms appearing in many documents.

Q. What is CountVectorizer?

Answer: A text vectorizer that represents documents using word or n-gram occurrence counts.

Q. Difference between CountVectorizer and TF-IDF?

Answer: CountVectorizer uses term counts, while TF-IDF weights terms using their frequency in a document and their distribution across documents.

Q. What is a unigram?

Answer: A single word considered as a feature.

Q. What is a bigram?

Answer: A pair of consecutive words considered as a feature.

Q. Why use ngram_range=(1, 2)?

Answer: To include both individual words and pairs of consecutive words as features.

Q. What does min_df=2 mean?

Answer: Ignore a term if it appears in fewer than two documents in the corpus used to fit the vectorizer.

Q. What does fit_transform() do?

Answer: Learns the vocabulary and IDF weights from the training data and transforms that data into a feature matrix.

Q. Why use transform() on test data?

Answer: To apply the already learned training vocabulary and IDF weights to test messages without refitting.

Q. Why split data into training and testing sets?

Answer: To train on one portion and evaluate performance on unseen data.

Q. Why use an 80:20 split?

Answer: It is a common choice that allocates most data for learning while keeping a separate test portion.

Q. What does random_state=42 do?

Answer: Makes the random split reproducible, assuming the same data and setup. The number 42 is simply a chosen seed.

Q. What does stratify=df["y"] do?

Answer: Maintains approximately the same class proportions in the training and testing sets.

Q. What is Naive Bayes?

Answer: A probabilistic classifier based on Bayes' theorem that assumes features are conditionally independent given the class.

Q. What is Bayes' theorem?

Answer: $P(A|B) = P(B|A) \times P(A) / P(B)$, where it calculates the probability of A given B.

Q. What is alpha in MultinomialNB?

Answer: A smoothing parameter that helps prevent zero probabilities for features not observed in a class.

Q. What is SVM?

Answer: Support Vector Machine, a supervised algorithm that learns a decision boundary to separate classes.

Q. What is a hyperplane?

Answer: A decision boundary that separates classes in a feature space.

Q. What are support vectors?

Answer: Training points closest to the decision boundary; they influence where the boundary is placed.

Q. Why use Linear SVM for text?

Answer: Text TF-IDF representations are often high-dimensional and sparse, and linear SVMs can work effectively in this setting.

Q. Why compare two algorithms?

Answer: To see which classifier performs better on the chosen dataset and evaluation measures.

Q. What is accuracy?

Answer: The proportion of all predictions that are correct.

Q. What is precision?

Answer: Among messages predicted as spam, the proportion that are actually spam.

Q. What is recall?

Answer: Among actual spam messages, the proportion correctly identified as spam.

Q. What is F1-score?

Answer: The harmonic mean of precision and recall.

Q. What is a confusion matrix?

Answer: A table showing correct and incorrect predictions for each class.

Q. What is a false positive?

Answer: A genuine ham message incorrectly classified as spam.

Q. What is a false negative?

Answer: A spam message incorrectly classified as ham.

Q. Which is more important: precision or recall?

Answer: It depends on the application. High precision avoids blocking genuine messages; high recall catches more spam. Both matter.

Q. Why is accuracy alone not enough?

Answer: The dataset is imbalanced, so a model can achieve high accuracy by favouring the majority ham class while missing spam. Precision, recall and F1-score give more detail.

Q. What is overfitting?

Answer: When a model fits the training data too closely and performs poorly on unseen data.

Q. Which model is used for the final new-message prediction in this code?

Answer: Multinomial Naive Bayes. Both models are trained and evaluated, but the final new-message section selects Naive Bayes.

7. Comparison of the two practicals

Parameter	Practical 1: Text preprocessing	Practical 2: Spam filtering
Main objective	Prepare raw text for analysis.	Classify SMS messages as spam or ham.
Main library	NLTK	Pandas and Scikit-learn
Input	A sample paragraph.	A labelled SMS dataset.
Main techniques	Cleaning, tokenization, stop-word removal, stemming.	TF-IDF feature extraction and classification.
Algorithms	Porter and Lancaster stemmers.	Multinomial Naive Bayes and Linear SVM.
Output	Processed tokens, stems and statistics.	Predicted message classes and evaluation scores.
Task type	Text preprocessing.	Supervised machine learning classification.

Relationship: Both practicals are related to NLP. Practical 1 prepares text through preprocessing. Practical 2 represents text numerically and uses machine learning to classify it.

8. Short explanations to speak in the external viva

Practical 1: 30–45 second explanation

"In this practical, we perform text preprocessing using the NLTK library. First, we take an input paragraph and convert it into lowercase. Then we remove special characters using regular expressions. After that, we perform tokenization to split the paragraph into individual words. We remove stop words such as 'is', 'are' and 'the'. Finally, we apply Porter and Lancaster stemming algorithms and compare their results. We also calculate the number of tokens before and after preprocessing, the percentage reduction and the number of unique stems."

Practical 2: 30–45 second explanation

"In this practical, we perform SMS spam detection using machine learning. We use the SMS Spam Collection dataset from the UCI repository. First, we load the dataset and convert spam and ham labels into numerical values. Then we divide the data into training and testing sets. We use TF-IDF to convert text messages into numerical features. After that, we train two algorithms, Naive Bayes and SVM. Finally, we compare their performance using accuracy, precision, recall, F1-score and a confusion matrix. We also test the trained Naive Bayes model on new SMS messages."

9. Last-minute revision sheet

Concept	Remember
Practical 1 sequence	Cleaning → Tokenization → Stop-word removal → Stemming → Statistics
Practical 2 sequence	Dataset → Label encoding → Train-test split → TF-IDF → Training → Evaluation → Prediction
NLTK	Natural Language Toolkit; Python library for NLP.
Tokenization	Splitting text into tokens.
Stemming	Rule-based reduction to a stem; stem may not be a dictionary word.

Lemmatization	Finding a meaningful dictionary base form.
TF-IDF	Converts text to numerical term weights.
Naive Bayes	Probabilistic classifier based on Bayes' theorem.
SVM	Learns a separating hyperplane; support vectors influence the boundary.
Accuracy	(TP + TN) / Total
Precision	TP / (TP + FP)
Recall	TP / (TP + FN)
F1-score	Harmonic mean of precision and recall.
Confusion matrix	[[TN, FP], [FN, TP]] for labels [0, 1] in this program.

10. Common installation and execution issues

If Python reports "ModuleNotFoundError: No module named nltk", install NLTK in the same Python environment or notebook kernel used to run the practical:

```
python -m pip install nltk
```

For the spam filtering practical, install the required packages:

```
python -m pip install pandas scikit-learn
```

The SMS practical downloads the UCI dataset from the internet, so an internet connection is required. The NLTK resource downloads may also require internet access the first time they are run.

If a package is installed but the import still fails, check that the terminal or notebook is using the same Python interpreter/environment in which the package was installed.